

task_bridge

작업 보드와 단일 진실 공급원, 완벽한 양방향 동기화

요약

task_bridge는 Go 내에 구현된 범용적이며 분리형 양방향 작업/보드 동기화 엔진입니다. 프로젝트의 작업 가능 항목(**SQLite**) 단일 진실 공급원을 트래커 문서 및 원격 보드(첫 지원 대상: ClickUp, 향후 Jira/Linear 지원 예정)와 결정론적 최종 편집 우선, 드라이런 우선, 데이터 손상 방지 원칙에 따라 동기화합니다.

간략 설명

프로젝트와 무관한 Go 하위 모듈로, 작업 가능 항목(**SQLite**) 단일 진실 공급원 ↔ 트래커 문서 ↔ 원격 보드(**ClickUp** 우선 지원)를 양방향 동기화합니다. 결정론적 최종 편집 우선, 드라이런 우선, HMAC 검증 웹훅을 적용하며, 모든 인증 정보와 ID는 런타임에 소비자가 주입합니다.

상세 설명

모든 팀은 결국 동일한 작업을 두 가지 장부에 기록하게 됩니다. 하나는 실제 작업 상태—코드, 문서, 내부 데이터베이스—이고, 다른 하나는 관리자가 보는 보드(예: ClickUp)입니다. 양쪽 중 어느 한쪽이라도 수정되는 순간 두 장부는 어긋나기 시작하며, 이를 수동으로 맞추는 일은 지루하고 오류가 발생하기 쉬운 작업이라 누구도 제대로 해내지 못합니다. task_bridge는 이 격차를 없애기 위해 세 가지 표현—프로젝트의 작업 가능 항목(**SQLite**) 단일 진실 공급원, 트래커 문서, 원격 보드(첫 지원 보드는 ClickUp, 향후 Jira와 Linear 추가 예정)—을 하나의 시스템으로 간주하고 동기화합니다.

동기화는 결정론적(최종 편집 우선)이며, 드라이런 우선 방식으로 설계되어 데이터 손상이나 유실이 절대 발생하지 않으며, 어느 한쪽이 오래된 상태로 방치되는 일도 없습니다. 부주의한 동기화로 일주일 치 작업이 덮어쓰일 수 있는 환경에서 이러한 안전성은 핵심 가치입니다.

아키텍처 측면에서 이 모듈은 엄격한 하위 모듈로, 다른 프로젝트에 의해 소비되며 Go의 분리 원칙(\$11.4.28)에 따라 프로젝트와 완전히 독립적입니다. 프로젝트별 값은 전혀 포함하지 않으며, 모든 인증 정보, 보드/폴더 ID, 항목 키 필드, 데이터베이스 경로는 소비자가 런타임에 pkg/config.Config를 통해 주입합니다.

모듈은 명확한 계층 구조로 설계되어 있습니다. - CLI(reconcile/push/pull/resolve/status/conflicts/init) 및 장기 실행 데몬(웹훅 수신기 + 크론 동기화) - MIT 라이선스의 raksul/go-clickup를 기반으로 한 경량 클라이언트 래퍼 - 실시간 API 프로브를 통해 보드/폴더 URL을 ID로 변환하는 리졸버(URL 문법 추측 없음) - 로컬 작업 항목과 원격 작업 필드 간의 매핑 - 명시적 충돌 처리 결과를 제공하는 최종 편집 우선 동기화 엔진 - X-Signature HMAC-SHA256 검증을 수행하는 웹훅 수신기

성숙도에 대해서는 솔직합니다. 현재는 P1 단계로, 레이아웃, 인터페이스, 진입점, 분리 경계는 구축되었으나 동기화 로직과 실시간 ClickUp 호출은 아직 구현되지 않았습니다(모든 스텝은 명시적 '미구현' 오류를 반환하며, 허위 정보 제공 금지 규칙을 따릅니다).

개발 배경

팀은 실제 작업 상태를 코드나 문서에 기록하지만, 관리자는 ClickUp 같은 보드에서 상황을 파악합니다. 이 두 가지는 끊임없이 어긋납니다.

task_bridge는 이 둘을 하나의 시스템으로 통합하여, 결정론적이고 안전하게 동기화함으로써 어느 한쪽도 오래되거나 잘못된 상태로 남지 않도록 합니다.

콘텐츠

왜 혁신적인가

양방향 보드 동기화는 보통 일회성으로, 팀마다 제대로 구현하지 못한 채 하드코딩된 통합 방식으로 이루어집니다. task_bridge은 이를 재사용 가능하고 자격 증명 주입이 가능한 라이브러리로 재정의하며, 엄격한 데이터 안전성 보장을 기본으로 탑재했습니다. 드라이런 우선 실행, 결정론적 최종 편집 우선 적용, HMAC 검증 이벤트 등을 통해 어떤 프로젝트든 취약한 커넥터를 새로 작성하는 대신 설정 주입만으로 신뢰할 수 있는 보드 통합을 구현할 수 있습니다.

혁신적인 점

- 3중 양방향 동기화: SQLite SSoT ↔ 트래커 문서 ↔ 원격 보드
- 완전한 디커플링(\$11.4.28): 프로젝트 값 없음. 모든 설정은 런타임에 주입됨
- 라이브 API URL→ID 변환으로 취약한 URL 문법 파싱 대체
- 실시간 이벤트를 위한 HMAC-SHA256 검증 웹훅 수신

도전 과제와 해결 방안

- 세 소스 간 데이터 안전성: 결정론적 최종 편집 우선 적용, 드라이런 우선 실행, 명시적 충돌 결과로 해결
- 디커풀링을 통한 재사용성: pkg/config 주입 경계로 해결(프로젝트별 세부 사항 없음)
- 신뢰할 수 있는 보드 식별: 라이브 API 프로브를 통한 URL→ID 변환으로 해결
- 투명한 스캐폴딩: 미구현 스텝은 명시적 '미구현' 오류 반환(가짜 응답 없음)

기술 스택(이유와 방법)

- **Go** — 엔진, CLI(cmd/task_bridge), 데몬(cmd/task_bridged)
- **SQLite** — 작업 가능한 항목의 단일 진실 공급원(SSoT)
- **raksul/go-clickup(MIT)** — ClickUp 전송 계층 래퍼
- **HMAC-SHA256** — 웹훅 서명 검증
- **cron + 웹훅** — 데몬 조정 및 실시간 이벤트 수신
- **pkg/config** — 런타임 자격 증명/ID 주입 경계

상태 투명성: 이는 **P1** 스캐폴딩입니다. 동기화 로직은 아직 구현되지 않았습니다. 출시된 제품으로 오해하지 마세요.